ch01 머신러닝 시작하기

- Machine Learning with sklearn @ DJ,Lim
- update : 2026/01

학습 내용

- 머신러닝을 학습하기 위한 기본 라이브러리 이해 및 실습
- 머신러닝 모델 만들어보기

01 기본 개념 이해하기

- **샘플(sample) or 데이터 포인터(data point)** : 하나의 개체 또는 관측치로, 데이터셋의 행(row)에 해당
- 특성 or 속성(feature) : 샘플의 개별 속성으로, 데이터셋의 열(column)에 해당.

02 기본 라이브러리 이해하기

scikit-learn(사이킷 런)

- 오픈 소스입니다.
- 매우 인기 높고 독보적인 파이썬 머신러닝 라이브러리입니다.
- url : http://scikit-learn.org/stable/documentation
- 사용자 가이드 : https://scikit-learn.org/stable/user_guide.html

Numpy

- 파이썬으로 과학 계산을 하기 위한 꼭 필요한 패키지
- 다차원 배열을 위한 기능
- 선형 대수 연산 기능
- 푸리에 변환 같은 고수준 수학 함수와 유사 난수 생성기 기능
- python의 데이터 처리에 비해 매우 빠른 속도를 보여주어 pandas, 딥러닝 모델의 기본 자료형으로 사용
- JAX, CuPy 등과의 호환성을 통해 GPU 가속 지원 확대
- url : https://www.numpy.org/

Pandas

- 데이터 처리와 분석을 위한 파이썬 라이브러리
- url : https://pandas.pydata.org/

- R의 data.frame을 본떠 설계한 **Dataframe**이라는 데이터 구조를 기반으로 만들어짐.
- SQL 처럼 테이블 쿼리나 조인을 수행 가능함.
- xlsx, csv등의 다양한 파일과 데이터베이스에서 데이터를 읽어들 일 수 있음.

- https://pandas.pydata.org/pandas-docs/stable/getting_started/10min.html

PyTorch & TensorFlow

- 심층학습(딥러닝)을 위한 두 주요 프레임워크
- PyTorch가 연구 분야에서 주도권을 가지며 산업계에서도 빠르게 성장 중
- TensorFlow/Keras는 프로덕션 환경에서의 강점 유지
- URL: https://pytorch.org/ 및 https://www.tensorflow.org/

Matplotlib

- 파이썬 대표적인 과학 계산용 그래프 라이브러리

Hugging Face

- 트랜스포머 기반 모델과 대규모 언어 모델(LLM)을 위한 생태계
- 사전 훈련된 수천 개의 모델과 데이터셋 제공
- URL: https://huggingface.co/

SciPy

- SciPy(https://www.scipy.org/scipylib) 과학 계산용 함수를 모아놓은 파이썬 패키지.
- 고성능 선형대수 기능, 함수 최적화, 신호 처리, 특수한 수학 함수와 통계 분포 등
- 희소 행렬 기능

mglearn

- 파이썬 라이브러리를 활용한 머신러닝 저자가 작성한 라이브러리
- 기본 머신러닝 내용 설명을 위한 그래프(knn 알고리즘 설명 등)

새로운 흐름

- 벡터 데이터베이스(Pinecone, Milvus): 임베딩 저장 및 검색
- LangChain & LlamaIndex: LLM 애플리케이션 개발을 위한 프레임워크
- 가속 컴퓨팅(RAPIDS, JAX): GPU 활용 데이터 처리 및 모델 훈련

현대적 머신러닝 워크플로우

1. 문제 정의 및 데이터 수집
2. 탐색적 데이터 분석(EDA)과 특성 엔지니어링
3. 모델 개발 및 실험 관리
4. 모델 평가 및 해석
5. 모델 배포 및 모니터링
6. 지속적인 재학습 및 개선

이제 머신러닝은 단순한 알고리즘 적용을 넘어 전체 MLOps(머신러닝 운영) 파이프라인을 구축하고 관리하는 방향으로 발전하고 있습니다.

In [47]:
```
from IPython.display import display, Image
```

03 라이브러리 소프트웨어 버전 확인

> 라이브러리이름.__version__
> 라이브러리이름.version

In [48]:
```
import sys
print("파이썬 버전 :", sys.version)
```

```
파이썬 버전 : 3.13.5 | packaged by Anaconda, Inc. | (main, Jun 12 2025, 16:37:03) [MSC v.1929 64 bit (AMD64)]
```

In [49]:
```
import pandas as pd
print("판다스 버전 :", pd.__version__)
```

```
판다스 버전 : 2.2.3
```

In [50]:
```
import matplotlib
import numpy as np
import scipy as sp
```

직접 해보기1

- matplotlib, numpy, seaborn, scipy의 각각의 버전을 확인해 보자.

- 추가 라이브러리를 설치할 때 명령
  - pip install [라이브러리명]

04 머신러닝 모델을 위한 iris 데이터를 준비

데이터 : 붓꽃

- 종류 : setosa, versicolor, virginica
- 데이터 내용 : 붓꽃의 꽃잎과 꽃받침
- 우리가 해결하려고 하는 문제 : 데이터를 주고 붓꽃의 종류 예측하기

In [51]:
```
from IPython.display import Image

# 주피터 노트북에서 이미지 표시
display(Image(filename='img/iris_setosa01.png'))
```



Iris setosa
Photo: Radomil, CC BY-SA 3.0

Iris versicolor
Photo: Dlanglois, CC BY-SA 3.0

Iris virginica
Photo: Frank Mayfield, CC BY-SA 2.0

Source: Wikimedia Commons (adapted: cropped & resized). Licensed CC BY-SA.

용어 이해하기

- **클래스(class)** : 출력될 수 있는 값들. 붓꽃의 종류들, 붓꽃의 종류는 세 클래스 중 하나에 속한다.
  - 모델이 예측할 수 있는 범주형 결과값들의 집합
  - (구글 용어집) One of a set of enumerated target values for a label.
  - 예: 붓꽃 데이터셋에서는 세 가지 품종(setosa, versicolor, virginica)이 클래스에 해당
- **레이블(label)** :
  - 개별 데이터 포인트에 할당된 실제 클래스 값.
  - 데이터 포인트 하나(붓꽃 하나)에 대한 기대 출력. **특정 데이터 포인트에 대한 출력**
  - (구글 용어집) 지도학습에서 예에 대한 답변 또는 결과 부분

데이터 준비

- scikit-learn에서 제공하는 기본 데이터 자료형를 이용한다.
- load_iris가 반환한 iris객체는 파이썬의 딕셔너리(Dictionary)와 유사한 Bunch 클래스의 객체

In [53]:
```
# ✅ 데이터 불러오기
from sklearn.datasets import load_iris
iris = pd.DataFrame(load_iris().data,
                    columns=load_iris().feature_names)
iris['target'] = load_iris().target
iris.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column             Non-Null Count  Dtype  
---  ------             --------------  -----  
 0   sepal length (cm)  150 non-null    float64
 1   sepal width (cm)   150 non-null    float64
 2   petal length (cm)  150 non-null    float64
 3   petal width (cm)   150 non-null    float64
 4   target             150 non-null    int64  
dtypes: float64(4), int64(1)
memory usage: 6.0 KB
```

In [54]:

```python
# 1. 기본 정보 확인
print(type(iris))
print(iris.keys())

# 2. 데이터 차원 및 메모리 확인
print(f"\n데이터 형태: {iris.shape}")
print(f"메모리 사용량: {iris.memory_usage().sum() / 1024:.2f} KB")

# 3. 간단한 탐색적 분석
print("\n=== 데이터 미리보기 ===")
print(iris.head())

print("\n=== 기술통계량 ===")
print(iris.describe())

print("\n=== 데이터 타입 ===")
print(iris.dtypes)

print("\n=== 결측치 확인 ===")
print(iris.isnull().sum())
```

```
<class 'pandas.core.frame.DataFrame'>
Index(['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)',
       'petal width (cm)', 'target'],
      dtype='object')

데이터 형태: (150, 5)
메모리 사용량: 5.99 KB

=== 데이터 미리보기 ===
   sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)
\
0                5.1               3.5                1.4               0.2
1                4.9               3.0                1.4               0.2
2                4.7               3.2                1.3               0.2
3                4.6               3.1                1.5               0.2
4                5.0               3.6                1.4               0.2

   target
0       0
1       0
2       0
3       0
4       0

=== 기술통계량 ===
       sepal length (cm)  sepal width (cm)  petal length (cm)  \
count         150.000000        150.000000         150.000000
mean            5.843333          3.057333           3.758000
std             0.828066          0.435866           1.765298
min             4.300000          2.000000           1.000000
25%             5.100000          2.800000           1.600000
50%             5.800000          3.000000           4.350000
75%             6.400000          3.300000           5.100000
max             7.900000          4.400000           6.900000

       petal width (cm)      target
count        150.000000  150.000000
mean           1.199333    1.000000
std            0.762238    0.819232
min            0.100000    0.000000
25%            0.300000    0.000000
50%            1.300000    1.000000
75%            1.800000    2.000000
max            2.500000    2.000000

=== 데이터 타입 ===
sepal length (cm)    float64
sepal width (cm)     float64
petal length (cm)    float64
petal width (cm)     float64
target                 int64
dtype: object

=== 결측치 확인 ===
sepal length (cm)    0
sepal width (cm)     0
```

```
petal length (cm)    0
petal width (cm)     0
target               0
dtype: int64
```

05 데이터를 훈련 데이터와 테스트 데이터로 나누기

- 우리는 모델을 만들고 모델의 성능 측정을 위해 학습용 데이터 셋과 테스트용 데이터 셋으로 나누어 준다.
- 학습용 데이터 : 모델 선택 후, 실제 학습에 사용하는 데이터 셋(실제 문제지의 문제)
- 테스트 데이터 : 학습 후, 실제 잘 동작하는지 확인하기 위한 데이터 (모의고사시험)
- 내용 : 모델을 새 데이터에 적용하기 전에 우리가 만든 모델이 잘 동작하는지 확인하기 위해 테스트 데이터를 활용하여 평가한다.

훈련 데이터 셋(training set) : 머신러닝 모델을 만들 때 쓰는 데이터 셋

테스트 데이터 셋(test set) : 모델이 얼마나 잘 작동하는지 쓰는 데이터 셋

- 테스트 데이터 셋을 다른 말로 홀드아웃(hold-out data)이라 한다.
  - 홀드 아웃 데이터 : 훈련중에 의도적으로 사용되지 않은 검증 데이터 세트 및 테스트 데이터 셋. 모델이 일반화 능력을 평가하는데 도움이 된다.

- scikit-learn은 데이터 셋을 나눠주기 위해 train_test_split 함수를 이용.
- train_test_split 함수는 기본적으로 75% 훈련 세트, 25%의 테스트 세트

In [55]:
```
iris.columns
```

Out[55]:
```
Index(['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)',
       'petal width (cm)', 'target'],
      dtype='object')
```

In [56]:
```
# 최신 방식: 훈련, 검증, 테스트 세트로 3단계 분할
from sklearn.model_selection import train_test_split

X = iris.loc[:, "sepal length (cm)":"petal width (cm)"]
y = iris['target']

# 먼저 테스트 세트 분리
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
                                                    random_state=42,
                                                    stratify=y)
```

In [57]:
```
# 데이터 사이즈
print(X_train.shape)   # 훈련 데이터 셋 사이즈
print(X_test.shape)    # 테스트 데이터 셋 사이즈
print(y_train.shape)   # 훈련 데이터 레이블 사이즈
print(y_test.shape)    # 테스트 데이터 레이블 사이즈
```

```
(120, 4)
(30, 4)
(120,)
(30,)
```

06. 데이터 살펴보기 - 시각화

- 머신러닝 모델을 만들기 전 머신러닝 없이도 풀 수 있는 문제는 아닌지, 혹은 필요한 정보가 누락이 없는지 확인
- 산점도(SCATTER PLOT)를 이용하여 확인.
  - 한특성을 x축에 놓고 다른 하나는 y축에 놓아 데이터를 점으로 나타내는 그래프
- 2개의 변수만 사용 가능하여 산점도 행렬(scatter matrix)를 사용

In [58]:
```
import seaborn as sns
```

In [59]:
```
iris.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column             Non-Null Count  Dtype  
---  ------             --------------  -----  
 0   sepal length (cm)  150 non-null    float64
 1   sepal width (cm)   150 non-null    float64
 2   petal length (cm)  150 non-null    float64
 3   petal width (cm)   150 non-null    float64
 4   target             150 non-null    int64  
dtypes: float64(4), int64(1)
memory usage: 6.0 KB
```

In [60]:
```
iris['target'] = iris['target'].astype('category') # 자료형 변환
iris.head()
```

Out[60]:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0

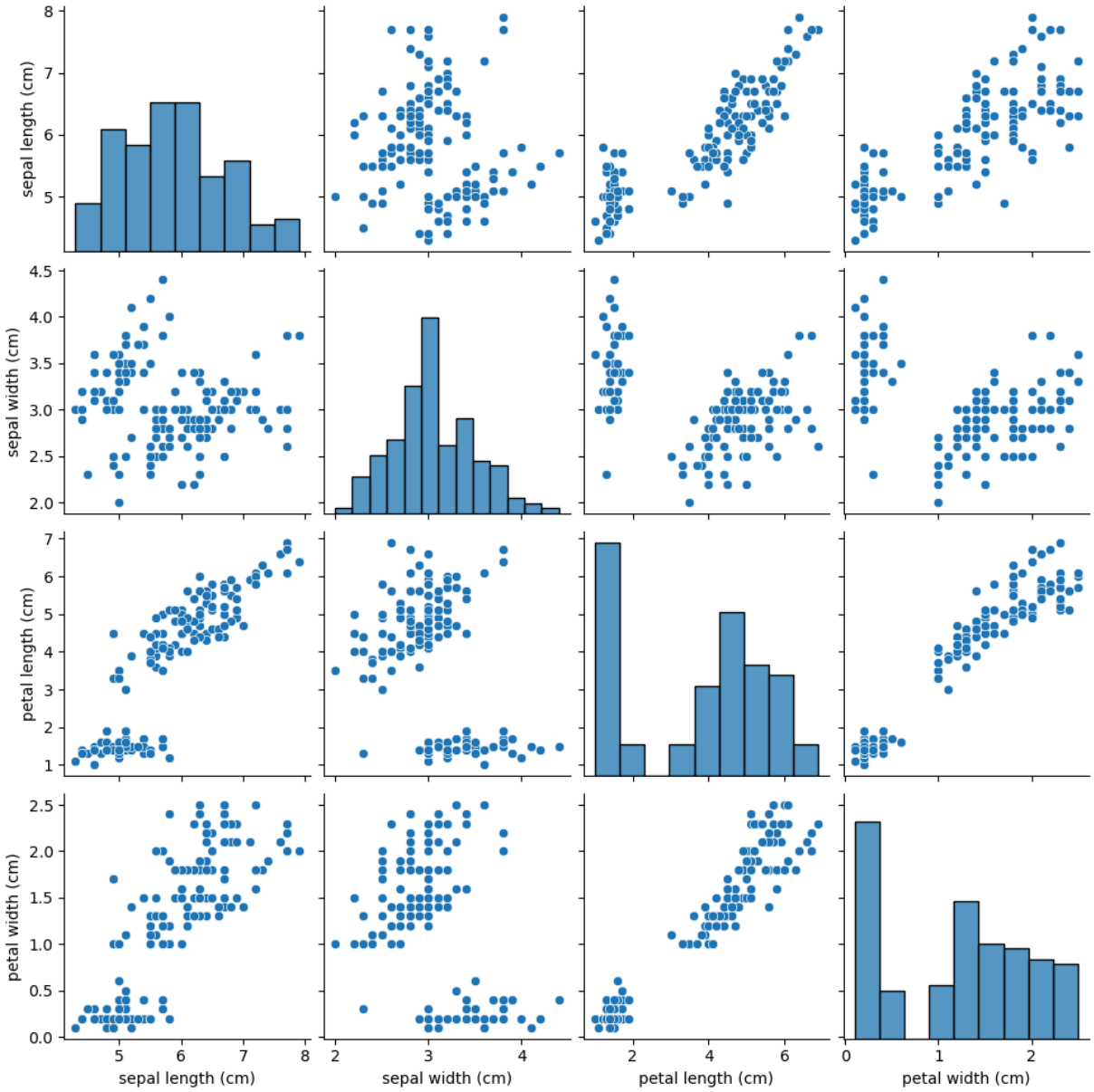
In [61]:
```
print(iris.shape)
print(iris.info())
```

```
(150, 5)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   sepal length (cm)  150 non-null    float64
 1   sepal width (cm)   150 non-null    float64
 2   petal length (cm)  150 non-null    float64
 3   petal width (cm)   150 non-null    float64
 4   target             150 non-null    category
dtypes: category(1), float64(4)
memory usage: 5.1 KB
None
```

seaborn를 활용한 산점도 행렬 확인

In [62]:
```
sns.pairplot(iris.iloc[ : ,0:4])   # 1~4열 선택
```

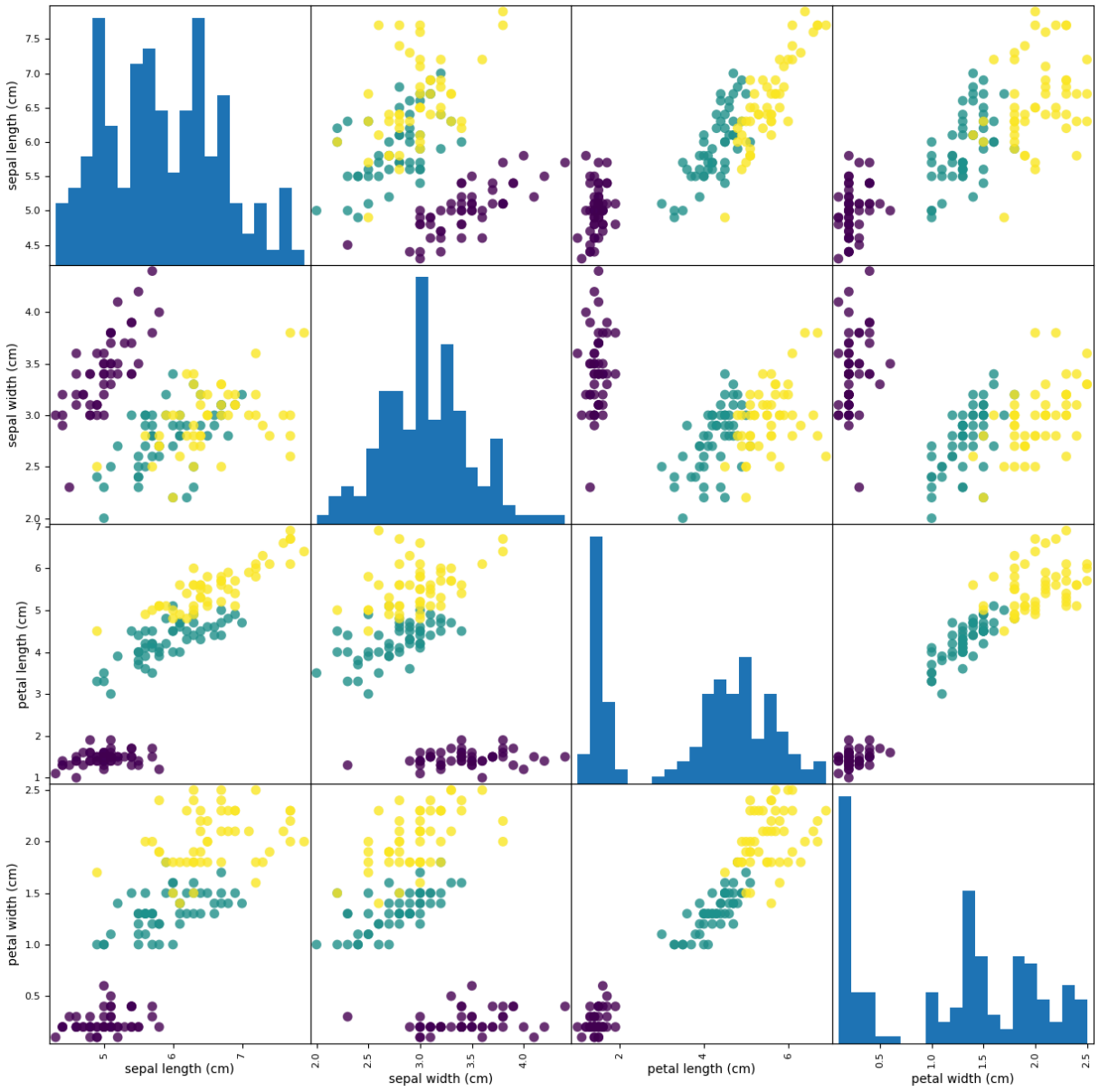
Out[62]:
```
<seaborn.axisgrid.PairGrid at 0x233b098fc50>
```



Pandas를 이용한 산점도 행렬 확인

In [63]:
```
pd.plotting.scatter_matrix(iris, c=y,      # 색
                           figsize=(15,15),          # 크기
                           marker='o',               # 표시
                           hist_kwds={'bins':20},    # 막대의 개수
                           s=60,      # size
                           alpha=0.8 )  # 투명도
```

```
Out[63]: array([[<Axes: xlabel='sepal length (cm)', ylabel='sepal length (cm)'>,
                 <Axes: xlabel='sepal width (cm)', ylabel='sepal length (cm)'>,
                 <Axes: xlabel='petal length (cm)', ylabel='sepal length (cm)'>,
                 <Axes: xlabel='petal width (cm)', ylabel='sepal length (cm)'>],
                [<Axes: xlabel='sepal length (cm)', ylabel='sepal width (cm)'>,
                 <Axes: xlabel='sepal width (cm)', ylabel='sepal width (cm)'>,
                 <Axes: xlabel='petal length (cm)', ylabel='sepal width (cm)'>,
                 <Axes: xlabel='petal width (cm)', ylabel='sepal width (cm)'>],
                [<Axes: xlabel='sepal length (cm)', ylabel='petal length (cm)'>,
                 <Axes: xlabel='sepal width (cm)', ylabel='petal length (cm)'>,
                 <Axes: xlabel='petal length (cm)', ylabel='petal length (cm)'>,
                 <Axes: xlabel='petal width (cm)', ylabel='petal length (cm)'>],
                [<Axes: xlabel='sepal length (cm)', ylabel='petal width (cm)'>,
                 <Axes: xlabel='sepal width (cm)', ylabel='petal width (cm)'>,
                 <Axes: xlabel='petal length (cm)', ylabel='petal width (cm)'>,
                 <Axes: xlabel='petal width (cm)', ylabel='petal width (cm)'>]],
               dtype=object)
```



07 첫번째 머신러닝 모델 만들기

- k-최근접 이웃(k-nearest neighbors, k-NN) 알고리즘 :
  - 훈련 데이터에서 새로운 데이터 포인트에 가장 가까운 'k개'의 이웃을 찾는다.
  - 이웃들의 클래스 중 빈도가 가장 높은 클래스를 예측값으로 사용

모델 만들기

In [64]:
```
from sklearn.neighbors import KNeighborsClassifier
model = KNeighborsClassifier(n_neighbors=2)  # 2개의 이웃을 기준
```

모델 학습시키기

In [65]:
```
model.fit(X_train, y_train)
```

Out[65]:

KNeighborsClassifier

```
KNeighborsClassifier(n_neighbors=2)
```

하나의 데이터로 예측해 보기

In [66]:
```
X_new = np.array([[5, 2.9, 1, 0.2]])
```

In [67]:
```
iris['target']
```

Out[67]:
```
0      0
1      0
2      0
3      0
4      0
      ..
145    2
146    2
147    2
148    2
149    2
Name: target, Length: 150, dtype: category
Categories (3, int64): [0, 1, 2]
```

In [68]:
```
### 예측시키기
pred = model.predict(X_new)
print("예측 : ", pred)
```

```
예측 :  [0]
c:\Users\daniel_wj\anaconda3\Lib\site-packages\sklearn\utils\validation.py:2739: UserWarning: X does not have valid feature names, but KNeighborsClassifier was fitted with feature names
  warnings.warn(
```

08 내가 만든 모델 평가하기

In [70]:
```
y_pred = model.predict(X_test)
print("예측값 :\n", y_pred)
```

```
예측값 :
 [0 2 1 1 0 1 0 0 2 1 2 2 2 1 0 0 0 1 1 1 0 2 1 1 2 1 1 0 2 0]
```

In [71]:
```
print("테스트 세트의 정확도 : {:.2f}".format(np.mean(y_pred == y_test)))
```

```
테스트 세트의 정확도 : 0.93
```

실습해 보기

- titanic 데이터 셋을 활용하여 knn 모델을 구현해 보자

REF

- sklearn score 매개변수 : https://scikit-learn.org/stable/modules/model_evaluation.html
- 구글 용어집 : https://developers.google.com/machine-learning/glossary/

History

- 내용 확인 및 업데이트 2021/10 v11
- 최종 확인 업데이트 2026/01 v13